

Camera Event Routing to Dedicated Writers Using RabbitMQ in .NET

When would someone need this information?

This guide is helpful when building a scalable surveillance or video event system, where each camera device sends events to a dedicated writer /processor, and you need to ensure one-to-one mapping between the camera and writer in a pub-sub messaging architecture using RabbitMQ.

Step-by-step guide

Goal :

Ensure that each camera's events are always processed by the same writer instance, even in a distributed environment, using RabbitMQ's routing and .NET consumers.

1. Choose the right RabbitMQ Exchange type

- Use **Direct** exchange or **Topic** exchange for routing based on camera ID.
- Each writer binds to a specific routing key: e.g., cam-1, cam-2, etc.

2. Publisher (.NET - Camera Service)

- Use a routing key format: "cam-{cameraId}"

```
_channel.BasicPublish( exchange: "camera-events", routingKey: "cam-1", basicProperties: null, body: messageBody);
```

3. Consumer (.NET - Writer Service)

- Each writer binds to the queue using its assigned routing key:

```
_channel.QueueBind( queue: "writer-1-queue", exchange: "camera-events", routingKey: "cam-1");
```

- Consume from the queue:

```
var consumer = new EventingBasicConsumer(_channel); consumer.Received += (model, ea) => { var body = ea.Body.ToArray(); // Process camera event };
_channel.BasicConsume("writer-1-queue", true, consumer);
```

4. Scale for 1000 cameras and writers

- Dynamically create queues per writer and bind them to the corresponding camera routing keys.
- Can be registered at startup using configuration (e.g., dictionary of cameraId to writerId).

Related Patterns

Use Case	Exchange Type	Routing Mechanism
One-to-One	Direct	routingKey = cam-1
One-to-Many	Fanout	All writers receive every event
Conditional Routing	Topic	cam.1.event, cam.*.error, etc.

Metadata-based Routing	Headers	Match based on headers like {"cameraId": "cam-1"}
------------------------	---------	---

Demo: Camera Event Routing System

This project demonstrates a RabbitMQ-based event routing system for multiple camera sources and interested services.

Project Structure

- EdgeGatewayAgent – Simulates camera event generation and publishes events via RabbitMQ.
- NVR1Service – Subscribes to "CAMERA-001" routing key.
- NVR2Service – Subscribes to "CAMERA-002" routing key.
- Storage1Service – Subscribes to "CAMERA-001" routing key.
- Storage2Service – Subscribes to "CAMERA-002" routing key.
- camera-event-visualizer – Vite + React frontend for visualizing event flow (optional).

Getting Started

Github Repo :-

1. Camera-Writer-RabbitMQ Header Exchange - Dockerize
2. Camera-Writer-RabbitMQ Direct Exchange

1. Start RabbitMQ

```
docker run --hostname rabbit --name rabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

RabbitMQ Management UI

Access it here: <http://localhost:15672>

Username: guest

Password: guest

2. Start All Services

In separate terminals :

```
# Terminal 1 - EdgeGatewayAgent
dotnet run --project EdgeGateway

# Terminal 2 - NVR1Service
dotnet run --project NVR1Service

# Terminal 3 - Storage1Service
dotnet run --project Storage1Service

# Terminal 4 - NVR2Service
dotnet run --project NVR2Service

# Terminal 5 - Storage2Service
dotnet run --project Storage2Service

# Terminal 6 - Web UI (optional)
cd camera-event-visualizer
npm install
npm run dev
```

Expected Behavior

- EdgeGatewayAgent emits camera events for CAMERA-001 and CAMERA-002.
- Events are published to a direct exchange with routing keys matching.
- Consumers (NVR/Storage services) receive only relevant events.
- No cross-contamination between camera routes.
- Web UI can be used to visualize routing in real time.